

# Lógica de Programação

Prof.<sup>a</sup> Kecia Aline Marques Ferreira

**DECOM** | Departamento de  
Computação



# Lógica de Programação

**Registros**

# Registros

- Conceito
- Definição de struct
- Referência a elementos da struct
- Atribuição de estruturas
- Matrizes de struct
- Exemplos

# Conceito

# Conceito

- Considere o caso a seguir:
  - Em um sistema hospitalar, os dados de um paciente são: cpf, rg, nome, endereço, telefone, altura e peso.
- Para fazer um programa que lide com esses dados, pode-se declarar uma variável correspondente a cada um dos dados:

```
char cpf[11];  
char rg[9];  
char nome[50];  
char endereco[150];  
char telefone[15];  
float altura;  
float peso;
```

# Conceito

- Porém, isso prolifera variáveis no programa. Para contornar esse problema, é importante a existência de um **recurso** de programação que permita agrupar todos esses dados e referenciá-los conjuntamente como Paciente

| Paciente                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>char cpf[11];<br/>char rg[9];<br/>char nome[50];<br/>char endereco[150];<br/>char telefone[15];<br/>float altura;<br/>float peso;</pre> |

# Conceito

- Esse recurso é denominado registro
- **Registro** é um conjunto de dados logicamente relacionados referenciado por um nome
- É uma **estrutura de dados heterogênea**, ou seja, pode envolver dados de tipos distintos
- Em C, registros são denominados **estruturas**
- A palavra **struct** é usada para definir uma estrutura em C

# Definição de struct



# Definição de struct

- A sintaxe para se definir uma estrutura é:

```
struct nome_da_struct{  
    lista de variáveis da struct  
};
```

Exemplo:

```
struct Paciente{  
    char cpf[11];  
    char rg[9];  
    char nome[50];  
    char endereco[150];  
    char telefone[15];  
    float altura;  
    float peso;  
};
```

# Definição de struct

- Observe que a definição de uma struct serve como uma forma, um modelo, para se criar variáveis do tipo dela posteriormente
- Ou seja, ao definir uma struct, nenhuma variável é criada de fato na memória.
- No exemplo, para se ter um registro de um paciente declarado de fato no programa, é preciso declarar uma variável do tipo Paciente

# Definição de struct

- Há duas formas de se declarar variáveis do tipo de uma estrutura

1) Junto com a definição da struct

```
struct nome_da_struct{  
    lista de variáveis da struct  
} declaração das variáveis;
```

Neste caso, não é necessário colocar o nome da estrutura

```
struct {  
    lista de variáveis da struct  
} declaração das variáveis;
```

# Definição de struct

## Exemplo

```
struct{  
    char cpf[11];  
    char rg[9];  
    char nome[50];  
    char endereco[150];  
    char telefone[15];  
    float altura;  
    float peso;  
}paciente1, paciente2;
```

```
struct Paciente{  
    char cpf[11];  
    char rg[9];  
    char nome[50];  
    char endereco[150];  
    char telefone[15];  
    float altura;  
    float peso;  
}paciente1, paciente2;
```

# Definição de struct

2) Após a definição da struct

```
struct nome_da_struct lista_de_variáveis;
```

Exemplo:

```
struct Paciente paciente1, paciente2;
```

Neste caso, a struct `Paciente` precisa ter sido definida anteriormente no código do programa.

# Definição de struct

- Em ambos casos, após a declaração das variáveis `paciente1` e `paciente2`, é reservada uma área de memória para cada uma delas

| <b>paciente1</b> |
|------------------|
| cpf              |
| rg               |
| nome             |
| endereco         |
| telefone         |
| altura           |
| peso             |

| <b>paciente2</b> |
|------------------|
| cpf              |
| rg               |
| nome             |
| endereco         |
| telefone         |
| altura           |
| peso             |

# Acesso a Elementos da struct

# Acesso a Elementos da struct

Os elementos de uma struct são referenciados utilizando-se o operador ponto (.)

Por exemplo, os comandos a seguir referenciam o elemento `peso`, `nome` e `altura` da variável `paciente1`, que é do tipo `struct Paciente`

```
paciente1.peso = 65.5;  
gets(paciente1.nome);  
printf("%f", paciente1.altura);
```



# Atribuição de struct

# Atribuição de struct

As informações contidas em uma variável do tipo struct podem ser atribuídas a outra do mesmo tipo

```
#include <stdio.h>

void main(){

    struct Ponto{
        float x;
        float y;
    };

    struct Ponto a,b;
    a.x=10;
    a.y=5;
    printf("O ponto a e (%f,%f).",a.x,a.y);

    b = a;
    printf("\nO ponto b e (%f,%f).",b.x,b.y);
}
```

# Matrizes de struct

# Matrizes de struct

Da mesma forma que se pode declarar uma matriz de um tipo atômico qualquer (int, float, etc.), pode-se declarar uma matriz de struct

Por exemplo, suponha o caso de um programa em que seja necessário ler e armazenar os dados de 10 pacientes

Para isso, pode-se criar um vetor (matriz unidimensional) de 10 posições do tipo Paciente

A declaração de uma matriz de struct é similar ao de um outro tipo qualquer, exceto que é necessário indicar que se trata de uma matriz, ou seja, usar os colchetes

# Matrizes de struct

Exemplo de declaração de matriz de struct junto com a definição da struct

```
struct{  
    char cpf[11];  
    char rg[9];  
    char nome[50];  
    char endereco[150];  
    char telefone[15];  
    float altura;  
    float peso;  
}pacientes[10];
```

```
struct Paciente{  
    char cpf[11];  
    char rg[9];  
    char nome[50];  
    char endereco[150];  
    char telefone[15];  
    float altura;  
    float peso;  
}pacientes[10];
```

# Matrizes de struct

Exemplo de declaração de matriz de struct após a definição da struct

```
struct nome_da_struct lista_de_variáveis;
```

Exemplo:

```
struct Paciente pacientes[10];
```

Neste caso, a struct `Paciente` precisa ter sido definida anteriormente no código do programa.

# Exemplo

# Exemplo

Um programa para ler, armazenar e imprimir os seguintes dados de 10 docentes: nome, titulação e ano de titulação.



# Exemplo

```
#include <stdio.h>
#define QTD 10

void main(){

    int i;

    struct Docente{
        char nome[100], titulacao[20];
        int anoTitulacao;
    }docentes[QTD];
```

# Exemplo

```
for (i=0; i<QTD; i++){
    printf("\n\nNome: ");
    gets(docentes[i].nome);
    printf("\nTitulacao: ");
    gets(docentes[i].titulacao);
    printf("\nAno titulacao: ");
    scanf("%d", &docentes[i].anoTitulacao);
    fflush(stdin);
}

printf("\n\n Os dados informados sao: ");

for (i=0; i<QTD; i++){
    printf("\n\nNome: %s", docentes[i].nome);
    printf("\nTitulacao: %s", docentes[i].titulacao);
    printf("\nAno titulacao: %d", docentes[i].anoTitulacao);
}

}
```